

Fungsi bernilai tabel

Mira Musrini Barmawi

TOPIK PEMBAHASAN

1. Inline table valued Function
2. Variable table
3. Multistatement Tabled Valued Function

Fungsi bernilai tabel

Fungsi bernilai tabel dalam SQL server 2012 ada dua jenis , yaitu :

- a. Inline table valued function
- b. Multi Statement Tabled Valued Function

Inline table valued function

Selanjutnya disebut sebagai fungsi bernilai tabel inline . Disingkat ITVF

Fungsi ini mengembalikan nilai dalam bentuk tabel. Cara penulisan fungsi mirip dengan View. Tidak ada blok begin...end. Pada fungsi ini tidak dapat dibuat program seperti kalimat IF dan kalimat pengulangan.

Multi Statement Tabled Valued Function

Selanjutnya disebut dengan fungsi bernilai tabel multistatement , atau disingkat MSTVF

Fungsi ini mengembalikan nilai dalam bentuk tabel . Perbedaannya dengan ITVF adalah , pada MSTVF , dapat dideklarasikan variabel , dibuat logika program seperti kalimat IF dan kalimat pengulangan (While ..do) ...

Pada MSTVF selalu ada blok begin end, dan tipe data yang dikembalikan berupa variable table .

Pada kuliah ini sebelum dibahas tentang MSTVF , kita bahas dulu tentang variable table

Inline Table Valued Function

Fungsi yang ditentukan pengguna bernilai tabel sebaris tidak memiliki isi BEGIN / END. Sebaliknya, pernyataan SELECT dikembalikan sebagai tabel virtual:

```
CREATE FUNCTION FunctionName (InputParameters)
RETURNS Table
AS
RETURN (Select Statement);
```

Inline table valued function- Example

```
CREATE FUNCTION dbo.fn_daftar_konsumen()  
RETURNS TABLE  
AS  
RETURN  
(SELECT * from customers)  
Go  
Select * from dbo.fn_daftar_konsumen()
```

Inline table valued function- Example

```
ALTER FUNCTION dbo.fn_daftar_konsumen(@id int)
RETURNS TABLE
AS
RETURN
(SELECT * from customers where customer_id=@id)
Go
Select * from dbo.fn_daftar_konsumen(3)
Go
Select * from dbo.fn_daftar_konsumen(4)
```


VARIABEL BERTIPE DATA TABEL

DEFINISI

Variabel tabel adalah tipe khusus dari variabel lokal yang membantu menyimpan data sementara, mirip dengan tabel temp di SQL Server. Faktanya, variabel tabel menyediakan semua properti dari variabel lokal, tetapi variabel lokal memiliki beberapa keterbatasan, tidak seperti tabel temp atau biasa.

DEKLARARASI VARIABEL BERTIPE DATA TABEL

```
DECLARE @LOCAL_TABLEVARIABLE TABLE  
(column_1 DATATYPE,  
    column_2 DATATYPE,  
    column_N DATATYPE  
)
```

VARIABEL BERTIPE DATA TABEL - CONTOH

```
go
```

```
DECLARE @ListOWeekDays TABLE (DyNumber INT, DayAbb VARCHAR(40) , WeekName  
VARCHAR(40))
```

```
INSERT INTO @ListOWeekDays
```

```
VALUES
```

```
(1, 'Mon', 'Monday'),  
(2, 'Tue', 'Tuesday') ,  
(3, 'Wed', 'Wednesday') ,  
(4, 'Thu', 'Thursday'),  
(5, 'Fri', 'Friday'),  
(6, 'Sat', 'Saturday'),  
(7, 'Sun', 'Sunday')
```

```
SELECT * FROM @ListOWeekDays
```

```
go
```

RESULT SET

Results		Messages	
	WeekDyNumber	WeekAbb	WeekName
1	1	Mon	Monday
2	2	Tue	Tuesday
3	3	Wed	Wednesday
4	4	Thu	Thursday
5	5	Fri	Friday
6	6	Sat	Saturday
7	7	Sun	Sunday

VARIABEL BERTIPE DATA TABEL - CONTOH

```
go
DECLARE @ListOWeekDays TABLE (DyNumber INT, DayAbb VARCHAR(40) , WeekName
VARCHAR(40))

INSERT INTO @ListOWeekDays
VALUES
(1, 'Mon', 'Monday') ,
(2, 'Tue', 'Tuesday') ,
(3, 'Wed', 'Wednesday') ,
(4, 'Thu', 'Thursday'),
(5, 'Fri', 'Friday'),
(6, 'Sat', 'Saturday'),
(7, 'Sun', 'Sunday')

DELETE @ListOWeekDays WHERE DyNumber=1
UPDATE @ListOWeekDays SET WeekName='Saturday is holiday' WHERE DyNumber=6
SELECT * FROM @ListOWeekDays
go
```

RESULT SET

Results		Messages	
	DyNumber	DayAbb	Week Name
1	2	Tue	Tuesday
2	3	Wed	Wednesday
3	4	Thu	Thursday
4	5	Fri	Friday
5	6	Sat	Saturday is holiday
6	7	Sun	Sunday

Apa lokasi penyimpanan variabel tabel?

- Jawaban atas pertanyaan ini adalah - variabel tabel disimpan dalam database tempdb. Mengapa kami menggarisbawahi ini karena terkadang jawaban untuk pertanyaan ini adalah variabel tabel disimpan dalam memori, tetapi ini sama sekali salah. Sebelum membuktikan jawaban atas pertanyaan ini, kita harus mengklarifikasi satu masalah tentang variabel tabel.

MELIHAT TEMPDB

```
go
DECLARE @ExperimentTable TABLE
(
    TestColumn_1 INT, TestColumn_2 VARCHAR(40), TestColumn_3
    VARCHAR(40)
);
SELECT TABLE_CATALOG, TABLE_SCHEMA, COLUMN_NAME, DATA_TYPE
FROM tempdb.INFORMATION_SCHEMA.COLUMNS
WHERE COLUMN_NAME LIKE 'TestColumn%';

GO
SELECT TABLE_CATALOG, TABLE_SCHEMA, COLUMN_NAME, DATA_TYPE
FROM tempdb.INFORMATION_SCHEMA.COLUMNS
WHERE COLUMN_NAME LIKE 'TestColumn%';
go
```


MULTI STATEMENT TABLE VALUED FUNCTION- example

```
CREATE FUNCTION dbo.fn_contacts()  
    RETURNS @contacts TABLE (  
        customer_name VARCHAR(40),  
        customer_address VARCHAR(40)  
    )  
AS  
BEGIN  
    INSERT INTO @contacts  
    SELECT  
        Customer_name,  
        Customer_address  
    FROM  
        customers;  
  
    RETURN;  
END;  
select * from dbo.fn_contacts()
```

CONTOH KASUS

Anda diminta untuk membuat invoice order , dengan daftar sebagai berikut :

Id Customer, customer name, customer address, orderdate, product name, quantity , unit price, quantity * unit price (subtotal) , grand total.

Contoh kasus

Menghitung grand total yang harus dibayar konsumen tidak dapat sekaligus dilakukan, dalam kueri yang sama. Harus ada dua kueri, kueri pertama menghitung subtotal . Kemudian kueri kedua menghitung total keseluruhan dari subtotal yang dihasilkan kueri pertama.

SOLUSI 1

1. Kueri pertama disimpan di view
2. Grand total dibuat dengan menghitung jumlah subtotal dari view tersebut. Dengan menggunakan aggregate function `sum()`

Alternatif solusi

Solusi 2

1. kueri pertama disimpan dalam tabel variabel
2. Grand total dibuat dengan menghitung jumlah subtotal dari variabel table tersebut. Dengan menggunakan aggregate function `sum()`

Solusi 1 hanya bisa dilakukan dalam 2 batch , View harus dibuat dulu , dan disimpan secara permanen di storage.

Solusi 2 bisa dilakukan dalam 1 batch . Table variable adalah tabel sementara yang umurnya hanya sepanjang batch dijalankan. Tabel variabel tidak akan disimpan didalam memori storage.

Solusi 1

```
GO
CREATE VIEW dbo.v_subTotal
AS
select o.Customer_id, customer_name, customer_address, orderdate, Product_name,
quantity
,unit_price, quantity*unit_price as 'subtotal' from
customers C inner join Orders O on C.customer_id=O.customer_id
inner join orderline OL on O.order_id=OL.order_id
inner join products P on OL.product_id =P.product_id
where o.Customer_id = 4
GO
GO
SELECT * FROM dbo.v_subTotal
SELECT SUM(subtotal) AS 'Grand Total' FROM dbo.v_subTotal
```

Solusi 2

```
DECLARE @invoice TABLE
(customer_id INT, customer_name VARCHAR(40), customer_address VARCHAR(40),
orderdate DATE, product_name VARCHAR(40),
quantity INT, unit_price DECIMAL(6,2), subtotal DECIMAL(6,2))

INSERT INTO @invoice
select o.Customer_id, customer_name, customer_address, orderdate,
Product_name, quantity
,unit_price, quantity*unit_price as 'subtotal' from
customers C inner join Orders O on C.customer_id=O.customer_id
inner join orderline OL on O.order_id=OL.order_id
inner join products P on OL.product_id =P.product_id
where o.Customer_id = 4

SELECT * FROM @invoice
SELECT SUM(subtotal) AS 'Grand Total' FROM @invoice
GO
```

Solusi 2 dapat disimpan dalam store procedure

```
CREATE PROCEDURE dbo.SP_Invoice1
AS
BEGIN
DECLARE @invoice TABLE
(customer_id INT, customer_name VARCHAR(40), customer_address VARCHAR(40), orderdate
DATE, product_name VARCHAR(40),
quantity INT, unit_price DECIMAL(6,2), subtotal DECIMAL(6,2))

INSERT INTO @invoice
select o.Customer_id, customer_name, customer_address, orderdate, Product_name,
quantity
,unit_price, quantity*unit_price as 'subtotal' from
customers C inner join Orders O on C.customer_id=O.customer_id
inner join orderline OL on O.order_id=OL.order_id
inner join products P on OL.product_id =P.product_id
where o.Customer_id = 4

SELECT * FROM @invoice
SELECT SUM(subtotal) AS 'Grand Total' FROM @invoice
END
GO
```

Solusi 2 dapat disimpan dalam store procedure

```
EXECUTE dbo.SP_Invoice1
```


Solusi 2 dapat disimpan dalam store procedure

```
CREATE PROCEDURE dbo.SP_Invoice2(@id INT)
AS
BEGIN
DECLARE @invoice TABLE
(customer_id INT, customer_name VARCHAR(40), customer_address VARCHAR(40), orderdate
DATE, product_name VARCHAR(40),
quantity INT, unit_price DECIMAL(6,2), subtotal DECIMAL(6,2))

INSERT INTO @invoice
select o.Customer_id, customer_name, customer_address, orderdate, Product_name,
quantity
,unit_price, quantity*unit_price as 'subtotal' from
customers C inner join Orders O on C.customer_id=O.customer_id
inner join orderline OL on O.order_id=OL.order_id
inner join products P on OL.product_id =P.product_id
where o.Customer_id = @id

SELECT * FROM @invoice
SELECT SUM(subtotal) AS 'Grand Total' FROM @invoice
END
GO
```

Solusi 2 dapat disimpan dalam store procedure

```
EXECUTE dbo.SP_Invoice2 1
```

```
go
```

```
EXECUTE dbo.SP_Invoice2 1
```

KUIS

Buat database akademik1

Tabel Mahasiswa				
ID_Mahasiswa	NRP	NAMA	tanggal lahir	ID Wali
10	16-2019-010	BAHY TSANY R	2000-10-01	1
11	16-2019-011	FAKHRI MOCHAMAD AZHAR	2000-08-02	2
12	16-2019-012	M RANGGA PRIMA YUDA	2000-09-03	3
13	16-2019-013	ARI YUDA PRATAMA SIRAIT	2000-10-04	4
14	16-2019-016	DIKY AKMAL FAUZI	2000-10-05	1
15	16-2019-018	PRAMBUDHI WIBOWO PANDJI	2000-01-06	2
16	16-2019-019	ISTIFAH	2000-10-07	3

Tabel Wali	
ID_wali	Nama Wali
1	Mira Musrini M.T
2	Sofia Umaroh M.T.
3	Kurnia Ramadhan Putra M.T.
4	Nur Fitrianti M.T.

KUIS

Anda berada di basisdata akademik1 , jawablah pertanyaan-pertanyaan berikut:

- a. Buatlah fungsi bernilai skalar, yang menghasilkan tanggal lahir paling akhir .
- b. Buatlah fungsi bernilai skalar, yang menghasilkan tanggal lahir yang paling awal
- c. Buatlah kueri yang menghasilkan NRP , Mahasiswa yang lahir paling akhir (gunakan fungsi skalar nomor a) .
- d. Buatlah kueri yang menghasilkan NRP, mahasiswa yang lahir paling awal (gunakan fungsi skalar nomor b)
- e. Buatlah fungsi bernilai tabel (inline tabel) yang menghasilkan daftar NRP, nama mahasiswa, Nama wali

Pertanyaan tugas

f. Buatlah daftar NRP, nama mahasiswa, Nama wali di variable table @daftar

g. Buatlah fungsi bernilai tabel (multistatement tabled valued function) yang menghasilkan daftar

NRP, nama mahasiswa, Nama wali